

DetECCIÓN Digital en Banda Base

1. Introducción

En esta sesión ampliaremos el trabajo realizado anteriormente mediante **la implementación del detector digital** que trabaja con el demodulador para decidir qué símbolo ha sido transmitido con mayor probabilidad.

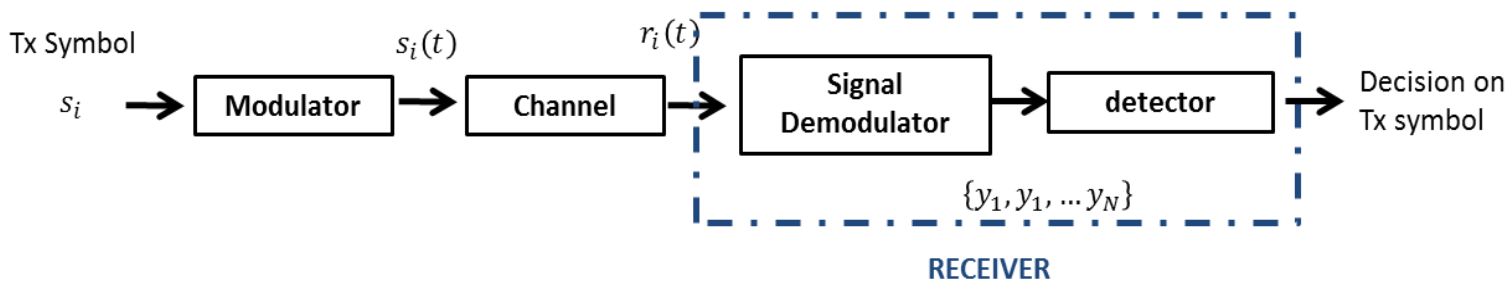


Figura 1 - Esquema completo del sistema de comunicación a implementar.

La sesión de laboratorio anterior se dedicó a la implementación de dos tipos de demoduladores diferentes. En la presente sesión nos centraremos en completar el resto de los bloques que se puede observar en la Figura 1. Le será útil centralizar todo en una función “main” que vaya llamando a otras funciones (tales como “Modulador” o “Demodulador”).

Una vez construido el sistema de comunicación, podremos estudiar su comportamiento.

2. Modulador

El bloque correspondiente al modulador digital en banda base se encarga de construir señales moduladas que varían en el tiempo, como una sucesión de símbolos. Los símbolos son el resultado de combinar linealmente ciertas bases ortogonales, siendo las amplitudes de la combinación lineal los bits de entrada al modulador.

En la presente sesión se utilizarán los símbolos (señales moduladas) que se detallan en la Figure 2.

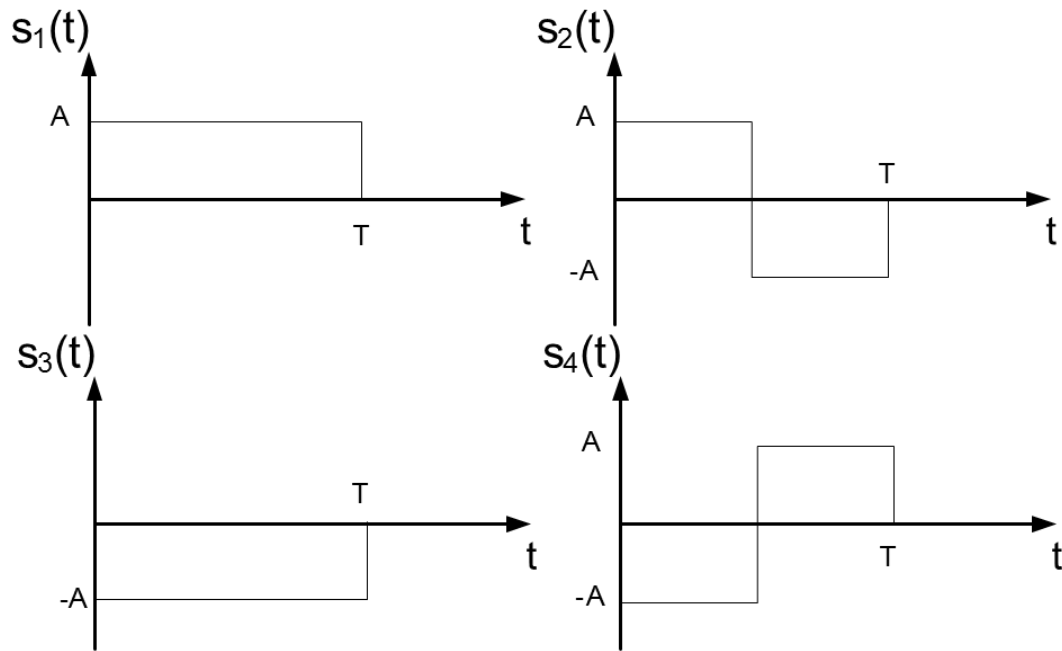


Figure 2 - Símbolos a utilizar.

En primer lugar, utilizando como funciones base $\phi_1(t), \phi_2(t)$, las empleadas en la práctica anterior, es decir, $\phi_1(t) = \frac{s_1(t)}{A\sqrt{T}}$, $\phi_2(t) = \frac{s_2(t)}{A\sqrt{T}}$, indique los coeficientes de las bases c_{ij} asociados a los símbolos $s_i(t)$ $i = 1, 2, 3, 4$ $j = 1, 2$. Tome $A = 1$, $T = 10$ ms.

Ejercicio 2.1 (2 puntos)

Implemente el bloque modulador. Como puede ver en la Figura 1, el bloque del modulador es un bloque que tiene como entrada la sucesión de bits a transmitir¹ y, como salida las correspondientes señales moduladas.

Desarrolle un código que implemente el comportamiento deseado, apoyándose si así lo cree conveniente en el código desarrollado en la práctica anterior. La función debe generar el vector de muestras que define la concatenación de varios símbolos, e incluir como parámetros de entrada al menos el número total de símbolos de salida, N , y los vectores de las funciones base ϕ_1, ϕ_2 . Por ejemplo, si $N = 10$, el vector de salida del modulador ha de contener la correspondiente señal modulada para estos 10 símbolos.

Para la generación del modulador asuma que $T = 10$ ms y utilice un periodo de muestreo de $T_s = T/20$, tal y como hizo en la sesión anterior.

¹ Siendo exactos, la sucesión s_i que se representa a la entrada es una sucesión de valores que van de 1 a 4, de modo que cada s_i representa dos bits. Por ejemplo, podríamos decir que la sucesión de bits de entrada es 0001101101..., y si s_1 representa a 00, s_2 representa a 01, s_3 representa a 10, y s_4 representa a 11, la sucesión de entrada será $s_1 s_2 s_3 s_4 s_2 \dots$ y el modulador se encargará de convertirla en $s_1(t)s_2(t)s_3(t)s_4(t)s_2(t) \dots$

Realice algunas pruebas para comprobar el correcto funcionamiento de la función. Muestre el resultado de alguna prueba, indicando el vector de entrada y representando gráficamente el de salida.

3. Demodulador

Ejercicio 3.1 (2 puntos)

Implementar el bloque demodulador basándose en el conjunto demodulador-muestreador de la sesión anterior. La entrada al bloque ha de ser el vector de muestras temporales de N símbolos, y la salida los vectores de las salidas de las dos ramas del correlador, muestreadas en los instantes $k \cdot T$, $k=1 \dots N$, es decir, cada vector de salida ha de tener una longitud N .

Realice alguna prueba, por ejemplo con $N=10$, mostrando que las salidas del demodulador se corresponden con los coeficientes de las funciones base asociados a los símbolos de entrada.

4. Detector

El detector recibe los vectores de salida del demodulador, y basándose en esos valores decide qué símbolo es más probable que fuera transmitido.

Ejercicio 4.1 (3 puntos)

Construya una función (`s_det = Detector (y)`) que reciba tantos parámetros de entrada como salidas tenga el demodulador implementado anteriormente. La misión de dicha función será decidir cuál de los símbolos fue transmitido.

Integre la función *Detector* con el resto de las funciones que ha implementado y haga algunas pruebas para asegurarse del correcto funcionamiento. Muestre y compare los símbolos detectados con los generados.

5. Probabilidad de error

A continuación, veremos cómo evaluar el comportamiento del sistema calculando la probabilidad de error de símbolo en función de la relación señal a ruido SNR.

Ejercicio 5.1 (3 Puntos)

Integre todos los bloques del sistema tal y como se muestra en la Figura 1, añadiendo ruido blanco a la señal de entrada al demodulador, y represente en una gráfica semilogarítmica la evolución de la probabilidad de error de símbolo en función de E_b/N_0 . El siguiente código incompleto puede que le sea de ayuda:

```

clear; close all, format compact;

M=4; % 4-symbol alphabet
EsN0_dB = 0:2:20; % Vector to simulate SNR
esn0_lin=10.^(EsN0_dB/10); % in linear units
ebn0_lin=esn0_lin/log2(M); % bit energy / noise linear units
EbN0_dB=10*log10(ebn0_lin); % bit energy / noise dBs
Nsymb = 10000; % Number of symbols
numErr = zeros(1,length(EsN0_dB)); % Pre-allocation

% Compute Pe for every SNR value
% ---- Tx ----
% Generate Nsymb random equiprobable symbols
s = randi(4,Nsymb,1);
% Modulate symbols
s_t = Modulador(s);

for iter_EsN0= 1:length(EsN0_dB)

    % ---- Channel ----
    % Compute symbols' power
    SNR_awgn_dB=EsN0_dB(iter_EsN0)+10*log10(Ts/T); % Compute
    corresponding SNR for the signal
    r_t = awgn(s_t,SNR_awgn_dB,'measured'); % Add noise

    % ---- Rx ----
    y = matchedFilter(r_t); % Or correlationType

    s_det = Detector(y); % Decided symbols

    % Compute number of wrong symbols comparing s_det with s
    numErr(iter_EsN0) = sum(s ~= s_det);

end

% Compute simulated Symbol Error Probability
Pe = numErr/Nsymb;

% Plot performance

% Show simulated performance
figure
semilogy(EbN0_dB,Pe,'o-')
grid on
xlabel('SNR [dB]')
ylabel('P_e')

% Compute theoretical performance
PeTheo = 2*qfunc(sqrt(ebn0_lin));
PeTheo(find(PeTheo<1e-7))=NaN;

hold on
semilogy(EbN0_dB,Pe,'x-')
semilogy(EbN0_dB,PeTheo)
legend('Simulada','Teorica','error probability')

```